МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №10
з дисципліни «Професійний Python»

Тема: «Production Python: Docker, CI/CD та захист проєкту»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Підготувати проєкт до production-oriented використання: оформити пакет (wheel і sdist), винести конфігурацію й секрети в environment variables, створити Docker image, налаштувати CI/CD у GitHub Actions і документацію.

1. Варіант 1 і структура проєкту

Варіант 1. Система обліку студентів. Lab_10 не створює нову систему, а переводить проєкт Lab_01–Lab_09 у production-стан. Вимоги варіанта й реалізація:

Таблиця 1 – Вимоги варіанта 1

Вимога	**Реалізація**
Продовжити проєкт Lab_01	студент: ім'я, прізвище, група, середній бал
Предметна функціональність	додавання/перегляд, пошук, фільтр за групою, найкращий студент, середній бал групи, перенесення між групами
REST API з Lab_08 + /health	`/students`, `/groups/{group_id}/students`, `/groups/{group_id}/statistics`, `GET /health`
Persistence Lab_07 і оптимізації Lab_09	SQLAlchemy + Alembic; кеш статистики груп з інвалідацією
Environment variables	`DATABASE_URL`, `ENVIRONMENT`, `LOG_LEVEL` (+ `API_TOKEN` як секрет)
Пакет	`student-manager` / `student_manager` 1.0.0

```
student-manager/
├── .github/workflows/ci.yml
├── src/student_manager/
│   ├── api.py            REST API, /health, захист записів
│   ├── config.py         Settings (environment variables)
│   ├── database.py       engine, сесії, міграції
│   ├── models.py  repositories.py  services.py  schemas.py
│   ├── caching.py        кеш статистики груп (після Lab_09)
│   ├── main.py           student-manager serve | migrate | config | ...
│   └── migrations/       Alembic (входять у wheel)
├── tests/                unit, integration, production-тести
├── tools/                check_secrets.py, smoke_test.py
├── Dockerfile  .dockerignore  .env.example  .gitignore
├── pyproject.toml  alembic.ini  LICENSE  README.md
```

2. Пакет: pyproject.toml, wheel, sdist

```
[project]
name = "student-manager"
version = "1.0.0"
description = "Student records REST API: FastAPI, SQLAlchemy, Alembic, Docker and CI/CD"
readme = "README.md"
requires-python = ">=3.11"
license = "MIT"
license-files = ["LICENSE"]
authors = [{ name = "Nazar Levandivskyi" }]
dependencies = [
    "fastapi>=0.115",
    "uvicorn>=0.30",
    "httpx>=0.27",
    "pydantic>=2.7",
    "pydantic-settings>=2.4",
    "sqlalchemy>=2.0",
    "alembic>=1.13",
]

[project.scripts]
student-manager = "student_manager.main:main"
student-api-client = "student_manager.external_api:main"
```

```
PS C:\Github\prof_python\Lab_10> python -m build
  - setuptools==84.2.1
* Building wheel...
```

```
PS C:\Github\prof_python\Lab_10> Get-ChildItem dist
Name                                 Length
----                                 ------
student_manager-1.0.0-py3-none-any.whl 467621
student_manager-1.0.0.tar.gz           493423
```

```
PS C:\Github\prof_python\Lab_10> twine check dist/*
Checking dist/student_manager-1.0.0-py3-none-any.whl: PASSED
Checking dist/student_manager-1.0.0.tar.gz: PASSED
```

Метадані зі збудованого wheel і потрібні для роботи файли (`migrations`, `static`, `py.typed`, `LICENSE`):

```
Name: student-manager
Version: 1.0.0
Summary: Student records REST API: FastAPI, SQLAlchemy, Alembic, Docker and CI/CD
License-Expression: MIT
Requires-Python: >=3.11
```

Файли wheel (вибірково):

```
student_manager/py.typed
student_manager/migrations/__init__.py
student_manager/migrations/env.py
student_manager/migrations/script.py.mako
student_manager/migrations/versions/0001_create_groups_and_students.py
student_manager/migrations/versions/0002_add_student_email.py
student_manager/migrations/versions/__init__.py
student_manager/static/swagger-ui-bundle.js
student_manager/static/swagger-ui.css
student_manager-1.0.0.dist-info/licenses/LICENSE
student_manager-1.0.0.dist-info/entry_points.txt
```

Перевірка wheel. `tools/smoke_test.py --wheel` створює чисте virtual environment, встановлює wheel (залежності з PyPI), запускає `student-manager serve` із production-змінними й перевіряє API; значення токена в конфігурації приховано:

```
PS C:\Github\prof_python\Lab_10> python tools/smoke_test.py --wheel dist\student_manager-1.0.0-py3-none-any.whl
student-manager 1.0.0
APP_NAME           Student Manager API
ENVIRONMENT        production
LOG_LEVEL          INFO
BATCH_CONCURRENCY  5
BATCH_TIMEOUT      10.0
API_TOKEN          **********
OK   GET /health: (200, 'ok')
    {"status": "ok", "environment": "production", "version": "1.0.0", "database": "ok"}
OK   POST /groups без X-API-Key: 401
OK   POST /groups: 201
OK   POST /students: 201
OK   GET /students?last_name=: (200, 1)
OK   GET /groups/1/statistics: (200, 1)
OK   GET /groups/{id}/students: (200, 1)
OK   DELETE /students/{id}: 204
OK   GET /openapi.json: 200
    маршрутів в OpenAPI: 9
БД збережена: True

[notice] A new release of pip is available: 25.0.1 -> 26.3
[notice] To update, run: C:\Users\Nazar\AppData\Local\Temp\student-manager-y456u8so\venv\Scripts\python.exe -m
pip install --upgrade pip
```

3. Конфігурація й секрети

```
class Settings(BaseSettings):
    """Налаштування застосунку з environment variables."""

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
        populate_by_name=True,
    )

    app_name: str = "Student Manager API"
    environment: Literal["development", "test", "production"] = Field(
```

```
        default="development",
        validation_alias=AliasChoices("ENVIRONMENT", "APP_ENV"),
    )
    database_url: str = "sqlite:///students.db"
    log_level: LogLevel = "INFO"
    batch_concurrency: int = Field(default=5, ge=1, le=64)
    batch_timeout: float = Field(default=10.0, gt=0, le=300)
    api_token: SecretStr | None = None

    @field_validator("log_level", mode="before")
    @classmethod
    def upper_level(cls, value: object) -> object:
        """Рівень журналу без урахування регістру: info -> INFO."""
        return value.upper() if isinstance(value, str) else value

    @field_validator("api_token", mode="before")
    @classmethod
    def empty_token_is_unset(cls, value: object) -> object:
        """API_TOKEN= у .env означає «не задано», а не порожній пароль."""
        if isinstance(value, str) and not value.strip():
            return None
        return value

    @property
    def safe_database_url(self) -> str:
        """URL бази для журналу: пароль замінено на ***."""
        return safe_url(self.database_url)

    @property
    def is_production(self) -> bool:
        """True у production-середовищі."""
        return self.environment == "production"

@lru_cache
def get_settings() -> Settings:
    """Налаштування читаються один раз за процес (зміна - перезапуск)."""
    return Settings()
```

```
PS C:\Github\prof_python\Lab_10> .env.example
APP_NAME=Student Manager API
ENVIRONMENT=development
DATABASE_URL=sqlite:///students.db
LOG_LEVEL=INFO
BATCH_CONCURRENCY=5
BATCH_TIMEOUT=10
API_TOKEN=
```

Секрети. У коді, README, Dockerfile і Git немає значень паролів чи токенів. `API_TOKEN` має тип `SecretStr` (у repr і журналах – зірочки), береться лише зі змінної середовища, локального `.env` або secrets платформи; у CI одноразовий токен створює `openssl rand`, у Docker він передається як `-e API_TOKEN` без значення в команді. Пароль у `DATABASE_URL` замінюється на *** у журналах. `.env` є в `.gitignore` і `.dockerignore`. Аудит:

```
PS C:\Github\prof_python\Lab_10> python tools/check_secrets.py
Перевірено файлів: 62; підозрілих рядків: 0
```

4. Docker

```
FROM python:3.12-slim AS builder
WORKDIR /build
RUN python -m pip install --no-cache-dir "build>=1.2"
COPY pyproject.toml README.md LICENSE ./
COPY src ./src
RUN python -m build --wheel --outdir /dist

FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1 \
    ENVIRONMENT=production \
    LOG_LEVEL=INFO \
    DATABASE_URL=sqlite:////app/data/students.db
WORKDIR /app
COPY --from=builder /dist/*.whl /tmp/
RUN python -m pip install /tmp/*.whl \
    && rm /tmp/*.whl \
    && useradd --create-home --uid 1000 appuser \
    && mkdir -p /app/data \
    && chown appuser:appuser /app/data
USER appuser
VOLUME ["/app/data"]
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=15s --retries=3 \
    CMD python -c "import sys, urllib.request;
sys.exit(urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2).status != 200)"
CMD ["student-manager", "serve", "--host", "0.0.0.0", "--port", "8000"]
```

```
PS C:\Github\prof_python\Lab_10> .dockerignore
.git
.github
.venv
.env
.env.*
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
.coverage
htmlcov
tests
tools
report
dist
build
*.egg-info
*.db
data
*.pdf
```

```
PS C:\Github\prof_python\Lab_10> docker build / docker run (команди з README)
docker build -t student-manager:1.0 .
docker run --rm -p 8000:8000 -e ENVIRONMENT=production \
    -e LOG_LEVEL=INFO -v student_data:/app/data student-manager:1.0
docker run -d -p 8000:8000 -e API_TOKEN -v student_data:/app/data student-manager:1.0
```

Docker на цьому комп'ютері не встановлено, тому образ збирає й запускає CI (runner Ubuntu): крок `Build image` будує образ за Dockerfile, контейнер стартує з volume, healthcheck стає healthy, процес працює від uid 1000, smoke-тест проходить, а після перезапуску на тому ж volume дані збережено. Ті самі production-змінні перевірено локально через wheel (розділ 2).

Таблиця 2 – Кроки Docker job у GitHub Actions

Крок	**Результат**	**Час**
Checkout	успішно	1 s
Generate a throwaway API token	успішно	0 s
Build image	успішно	19 s

Крок	Результат	Час
Show image	успішно	0 s
Run container with a volume	успішно	0 s
Wait for the Docker healthcheck	успішно	6 s
Container runs as a non-root user	успішно	0 s
Smoke test the API	успішно	1 s
Restart the container on the same volume	успішно	0 s
Data survived the restart	успішно	1 s
Job summary	успішно	0 s
Container logs	успішно	0 s
Remove container and volume	успішно	0 s

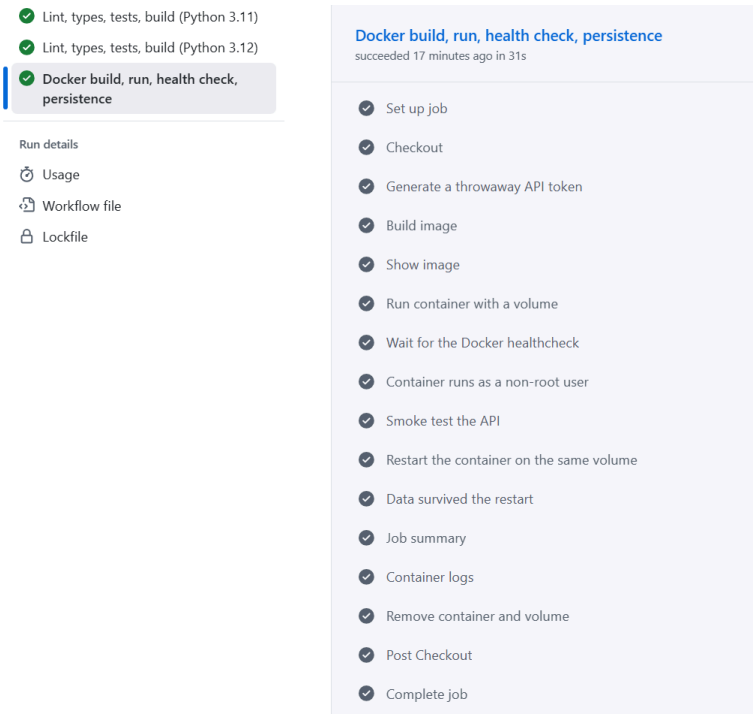


Рисунок 1 – Сторінка Docker job у GitHub Actions: усі кроки виконано (запуск #2)

5. REST API

Таблиця 3 – Основні endpoints

Endpoint	Призначення
GET /health	стан сервісу, версія, середовище, база даних
GET /students	пошук за прізвищем, фільтри, сортування, сторінки
POST /students	додати студента (потрібен X-API-Key за наявності токена)
GET /groups/{group_id}/students	студенти групи
GET /groups/{group_id}/statistics	кількість, середній, найкращий і найгірший бал
POST /groups/{group_id}/transfer	перенести студентів між групами

Результат `GET /health` (production-запуск із wheel, HTTP 200; у CI той самий запит виконує smoke-тест у контейнері):

```
PS C:\Github\prof_python\Lab_10> GET http://127.0.0.1:<port>/health
{"status": "ok", "environment": "production", "version": "1.0.0", "database": "ok"}
```

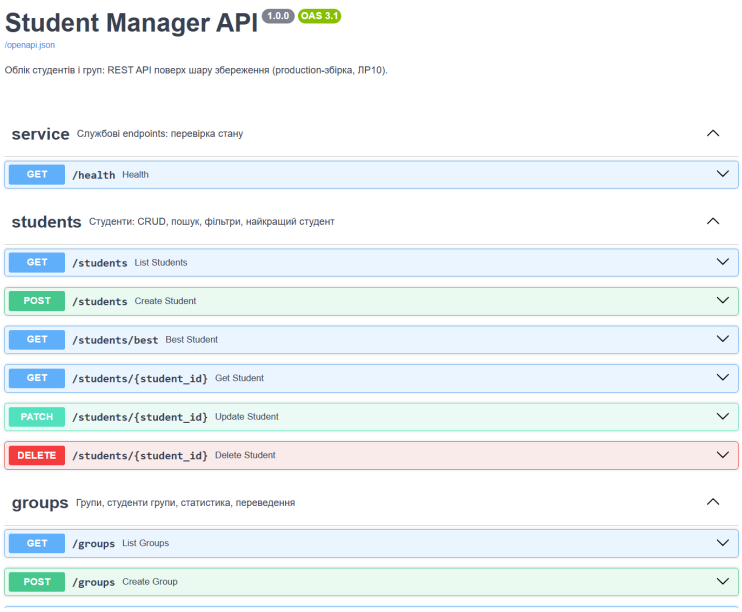


Рисунок 2 – Swagger UI (/docs) працюючого сервера, версія 1.0.0

6. CI/CD: GitHub Actions

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  quality:
    name: Lint, types, tests, build (Python ${{ matrix.python-version }})
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12"]
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip
          cache-dependency-path: pyproject.toml

      - name: Install
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"

      - name: Secret scan
        run: python tools/check_secrets.py

      - name: Ruff lint
        run: ruff check .

      - name: Ruff format check
        run: ruff format --check .

      - name: Type check (mypy --strict)
```

```
        run: mypy

      - name: Tests with coverage
        run: pytest -v --cov --cov-report=term-missing
```

Workflow (продовження): `python -m build`, `twine check`, встановлення wheel у чистому середовищі, артефакт dist; job `docker` (розділ 4) стартує після успіху `quality`.

Таблиця 4 – Запуск #2 (push у main, e1dd1ce): jobs якості на Python 3.11 і 3.12

Job	Результат	Час	Кроки (тривалість)
Lint, types, tests, build (Python 3.12)	успішно	60 s	Checkout 1 s; Set up Python 1 s; Install 13 s; Secret scan 0 s; Ruff lint 0 s; Ruff format check 0 s; Type check (mypy --strict) 5 s; Tests with coverage 24 s; Build wheel and sdist 3 s; Check distributions 0 s; Install the wheel in a clean virtual environment and run it 7 s; Upload package artifact 2 s
Lint, types, tests, build (Python 3.11)	успішно	76 s	Checkout 1 s; Set up Python 1 s; Install 16 s; Secret scan 0 s; Ruff lint 0 s; Ruff format check 0 s; Type check (mypy --strict) 9 s; Tests with coverage 27 s; Build wheel and sdist 3 s; Check distributions 0 s; Install the wheel in a clean virtual environment and run it 12 s; Upload package artifact 0 s (пропущено)

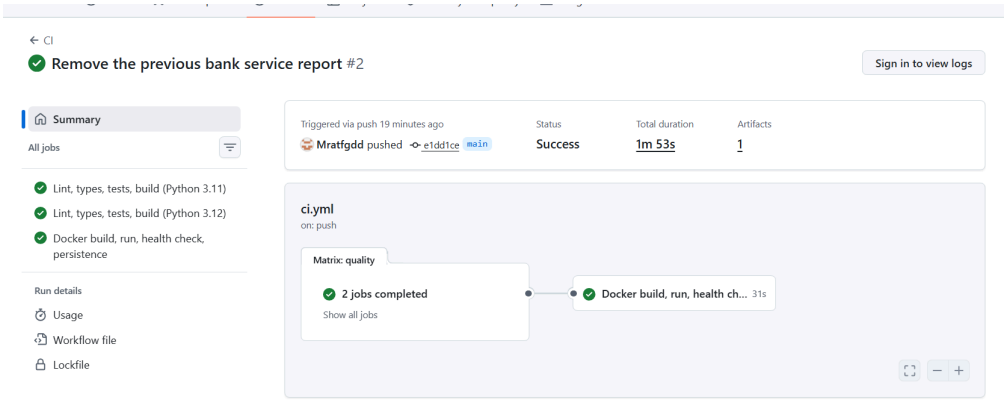


Рисунок 3 – Зелений pipeline: запуск #2 для коміту e1dd1ce

7. Якість: lint, типи, тести

```
PS C:\Github\prof_python\Lab_10> ruff check .
All checks passed!
```

```
PS C:\Github\prof_python\Lab_10> ruff format --check .
55 files already formatted
```

```
PS C:\Github\prof_python\Lab_10> mypy
Success: no issues found in 49 source files
```

```
PS C:\Github\prof_python\Lab_10> pytest -q --cov --cov-report=term-missing
Name                                    Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------
src\student_manager\__init__.py             1      0      0      0   100%
src\student_manager\api.py                162      0     16      0   100%
src\student_manager\caching.py             30      0      4      0   100%
src\student_manager\config.py              39      0      4      0   100%
src\student_manager\database.py            41      0      6      2    96%   37->39, 45->47
src\student_manager\dbapi.py               20      0      0      0   100%
src\student_manager\dependencies.py        65      0      8      0   100%
src\student_manager\exceptions.py          10      0      0      0   100%
src\student_manager\external_api.py       112      0     16      0   100%
src\student_manager\main.py               169      0     34      0   100%
src\student_manager\models.py              40      0      0      0   100%
src\student_manager\repositories.py        97      0     12      0   100%
src\student_manager\schemas.py             74      0      6      0   100%
src\student_manager\services.py           187      0     48      0   100%
src\student_manager\validation.py          33      0     14      0   100%
-----------------------------------------------------------------------------------
TOTAL                                    1080      0    168      2    99%
Required test coverage of 90.0% reached. Total coverage: 99.84%
526 passed in 54.71s
```

8. Git, README і production checklist

```
PS C:\Github\prof_python\Lab_10> git log --oneline --reverse
094d238 Rebuild the project as the student_manager REST API
a01f01d Add production configuration and Python packaging
809aa50 Add the test suite
2b34e58 Add Docker support
35e7ae4 Add CI pipeline and project tools
474ec1a Add project documentation
e1dd1ce Remove the previous bank service report
```

README описує систему, встановлення, конфігурацію, запуск, Docker, API, тести, CI/CD і сценарій демонстрації; розділи: `Можливості`, `Структура`, `Встановлення`, `Конфігурація`, `Запуск`, `Docker`, `API`, `Тести`, `Пакет`, `CI/CD`, `Демонстрація`, `Ліцензія`.

Таблиця 5 – Production checklist

Пункт		**Підтвердження**
tests pass	✓	526 passed, coverage 99.84 %
lint passes	✓	ruff check, ruff format --check
type checking passes	✓	mypy --strict
wheel builds	✓	dist/*.whl
sdist builds	✓	dist/*.tar.gz
wheel installs in a clean venv	✓	tools/smoke_test.py --wheel
Docker image builds	✓	CI: Build image
container starts and is healthy	✓	CI: healthcheck
/health returns 200	✓	wheel + CI
REST API works	✓	smoke-тест
.env is ignored	✓	.gitignore, .dockerignore
.env.example exists	✓	.env.example
no secrets in Git	✓	check_secrets.py
README updated	✓	README.md
CI pipeline green	✓	запуск #2
version defined	✓	1.0.0

Висновки

Проєкт «Система обліку студентів» переведено в production-oriented стан: пакет student-manager 1.0.0 (wheel і sdist), конфігурація через environment variables із секретом типу SecretStr, /health, багатоетапний Docker image з non-root користувачем, volume для SQLite і HEALTHCHECK, CI у GitHub Actions (запуск #2 зелений на Python 3.11 і 3.12, Docker job успішний). 526 тестів дають покриття 99,84 % при порозі 90 %. Обмеження: Docker не встановлено локально, тож образ збирається й запускається лише в CI.